

SUPPLEMENTARY FILES

This Supplementary Material accompanies the paper titled “*LLMs-Based Collaborative Evolutionary Framework for Heterogeneous Flexible Job-shop Scheduling with Cooperative Units*”, submitted to *Journal of Intelligent Manufacturing*. Due to the limited space available in the main manuscript, details regarding model constraints, framework implementation, pseudocode, and part of the experimental results are not included in the main text. To ensure that experts and readers have full access to the information, these details are provided here. The Supplementary Material is organized as follows:

- **Section 1:** Defines the constraints of the mathematical model.
- **Section 2:** Presents the framework details, including:
 - **Section 2.A:** Prompt structures responsible for framework evolution.
 - **Sections 2.B and 2.C:** Pseudocode for the framework and constraint checking.
 - **Section 2.D:** Predefined strategy modules.
- **Section 3:** Provides detailed experimental results, including:
 - **Section 3.A:** Data for hyperparameter selection.
 - **Section 3.B:** Detailed ablation study results and the Elo method.
 - **Section 3.C:** Comparative results against other algorithms.

1. Supplement to the Mathematical Model of HFJSCU

The objective function of HFJSCU is defined as follows:

$$\min C_{max} \quad (1)$$

subject to:

$$S_{i,j} \geq C_{i,j-1}, \quad \forall i \in \{1, \dots, N\}, j \in \{2, \dots, K_i\} \quad (2)$$

$$\sum_{k=1}^M x_{i,j,k} = 1, C_{i,j} = S_{i,j} + \sum_{k=1}^M x_{i,j,k} \cdot T_{i,j,k}, \quad \forall i \in \{1, \dots, N\}, j \in \{1, \dots, K_i\} \quad (3)$$

$$S_v^{prep} \geq C_u - V \cdot (1 - z_{u,v}) - V \cdot (2 - x_{u,k} - x_{v,k}), \quad \forall u, v \in O, u \neq v, \forall k \in \{1, \dots, M\} \quad (4)$$

$$S_u^{prep} \geq C_v - V \cdot z_{u,v} - V \cdot (2 - x_{u,k} - x_{v,k}), \quad \forall u, v \in O, u \neq v, \forall k \in \{1, \dots, M\} \quad (5)$$

$$x_{u,k} + x_{v,k} - 1 \leq z_{u,v} + z_{v,u}, \quad \forall u, v \in O, u \neq v, \forall k \in \{1, \dots, M\} \quad (6)$$

$$\sum_{l=1}^C y_{i,j,l} = N_p, \quad \forall i \in \{1, \dots, N\}, j \in \{1, \dots, K_i\} \quad (7)$$

$$S_{i,j,l}^{prep} \leq V \cdot y_{i,j,l}, C_{i,j,l}^{prep} \geq S_{i,j,l}^{prep} + T_{i,j,l}^{prep} - V \cdot (1 - y_{i,j,l}), \quad \forall i, j, l \quad (8)$$

$$C_{i,j,l}^{prep} \leq S_{i,j,l}^{prep} + T_{i,j,l}^{prep} + V \cdot (1 - y_{i,j,l}), C_{i,j,l}^{prep} \leq V \cdot y_{i,j,l}, \quad \forall i, j, l \quad (9)$$

$$C_{i,j}^{prep} = S_{i,j}^{prep} + \sum_{l=1}^C y_{i,j,l} \cdot T_{i,j,l}^{prep}, \quad \forall i, j \quad (10)$$

$$S_{i,j}^{prep} \geq S_{i,j-1} + \sum_{k=1}^M x_{i,j-1,k} \cdot V - V, \quad \forall i \in \{1, \dots, N\}, j \in \{2, \dots, K_i\} \quad (11)$$

$$S_{i,j,l}^{prep} \geq S_{i,j}^{prep} - V \cdot (1 - y_{i,j,l}), C_{i,j}^{prep} \leq C_{i,j,l}^{prep} + V \cdot (1 - y_{i,j,l}), \quad \forall i, j, l \quad (12)$$

$$S_u^{prep} + V \cdot (1 - w_{u,v}) \geq C_v^{prep}, \quad \forall u, v \in O, u \neq v, \forall l \in \{1, \dots, C\} \quad (13)$$

$$S_v^{prep} + V \cdot w_{u,v} \geq C_u^{prep}, \quad \forall u, v \in O, u \neq v, \forall l \in \{1, \dots, C\} \quad (14)$$

$$y_{u,l} + y_{v,l} - 1 \leq w_{u,v} + w_{v,u}, \quad \forall u, v \in O, u \neq v, \forall l \in \{1, \dots, C\} \quad (15)$$

$$S_{i,j} \geq C_{i,j}^{prep}, \quad \forall i \in \{1, \dots, N\}, j \in \{1, \dots, K_i\} \quad (16)$$

$$C_{max} \geq C_{i,K_i}, \quad \forall i \in \{1, \dots, N\} \quad (17)$$

$$S_{i,j}, C_{i,j}, S_{i,j}^{prep}, C_{i,j}^{prep}, C_{max} > 0 \quad (18)$$

The constraints define the feasible solution space of HFJSCU. Eq. (2) ensures that operations within each job are processed according to the constrained order. Eq. (3) guarantees the assignment of operations to permissible machines and calculates processing time. Eqs. (4) to (6) enforce that working time on machines has exclusivity. Eq. (7) manages the number of collaborative units required for each operation. Eqs. (8) and (9) accurately calculate the start and completion times of each collaborative unit. Eq. (10) to (12) determine the overall start and completion times of operation preparation. Eq. (13) to (15) prevent collaborative units from executing multiple tasks simultaneously. Eq. (16) enforces processing time to start after the necessary operation preparation is completed. Eq. (17) defines makespan as the maximum completion time of all jobs. Finally, Eq. (18) specifies the permissible values of mathematical variables.

2. Supplementary Content for LCEoMA

A. Introduction to Module Prompts

This section provides a detailed description of the evolutionary operators within the framework. The prompts designed for module individuals are referred to as Module Prompts B. The following text presents the five types of prompts used: [D, V, M, S, R].

Prompt Template B-D

- You are an expert in memetic algorithm design.
 - Below is the {module_name} strategy module for a memetic algorithm (information within triple quotes describes the module's design approach), intended to solve the flexible job-shop scheduling with cooperative units.
{target_code}
 - *Optimization Objectives*:
 - Generate an optimization method with a **completely different approach from the current one**, to replace the {model_name} code and reduce makespan.
 - *Output Requirements*:
 - 1. Maintain the existing interface (function names, parameters).
 - 2. Do not include any explanations.
 - 3. Enclose comments within triple quotes. The code should be wrapped in a Markdown code block, formatted as follows:
- ```
'''Approach description: xxx'''
```python
def xxx():
# ... your code here ...
...

```

Prompt Template B-V

- You are an expert in memetic algorithm design.
 - Below is the {module_name} strategy module for a memetic algorithm (information within triple quotes describes the module's design approach), intended to solve the flexible job-shop scheduling with cooperative units.
{target_code}
 - *Optimization Objectives*:
 - Generate an optimization method that **shares similarities with the current approach but offers distinct differences**, to replace the {model_name} code and reduce makespan.
 - *Output Requirements*:
 - 1. Maintain the existing interface (function names, parameters).
 - 2. Do not include any explanations.
 - 3. Enclose comments within triple quotes. The code should be wrapped in a Markdown code block, formatted as follows:
- ```
'''Approach description: xxx'''
```python
def xxx():
# ... your code here ...
...

```

Prompt Template B-M

- You are an expert in memetic algorithm design.
- Below is the overall framework for a memetic algorithm, intended to solve the flexible job-shop scheduling with cooperative units.
{target_code}
- Optimization Objectives: **Adjust only the algorithm's hyperparameters** to reduce makespan.
- Output Requirements: Keep other parts unchanged. Do not include any explanations. Output only the algorithm code within a Markdown code block.

Module Prompt B-S

- You are an expert in memetic algorithm design.
- Below is the {module_name} strategy module for a memetic algorithm (information within triple quotes describes the module's design approach), intended to solve the flexible job-shop scheduling with cooperative units.
{target_code}
- *Optimization Objectives*:
- **Simplify the current optimization strategy code to reduce its runtime**, to replace the {model_name} code and reduce makespan.
- *Output Requirements*:
- 1. Maintain the existing interface (function names, parameters).
- 2. Do not include any explanations.
- 3. Enclose comments within triple quotes. The code should be wrapped in a Markdown code block, formatted as follows:

```
'''Approach description: xxx'''
```python
def xxx():
... your code here ...
...

```

### Module Prompt B-R

- You are an expert in memetic algorithm design.
- Below is the {module\_name} strategy module for a memetic algorithm, intended to solve the flexible job-shop scheduling with cooperative units.  
{target\_code}
- \*Issue Present\*:
- The strategy module code causes runtime errors. **The first triple-quoted section contains the error cause and modification suggestions, while the second triple-quoted section describes the strategy module's design approach.**
- \*Optimization Objectives\*:
- **Retain the current strategy module's design approach; only correct the errors.**
- \*Output Requirements\*:
- 1. Maintain the existing interface (function names, parameters).
- 2. Do not include any explanations.
- 3. Delete the content of the first triple-quoted section, retaining only the method description from the second triple-quoted section and the corrected code. The code should be wrapped in a Markdown code block, formatted as follows:  

```
'''Approach description: xxx'''
```python
def xxx():
# ... your code here ...
...

```

B. Description of LCEoMA

Algorithm 1: LCEoMA

Input: Number of Iterations ($MaxIter$), population size (ps), maximum temperature ($T_{\max}$), LLM score (LLM_s), Minimum Probability for LLM Selection (ϵ_{LLM}), Minimum Probability for Individual Selection (ϵ_M), images, prompt A (P_A), module prompt B (P_B), baseline module (BM)

Output: Excellent modules ($\mathcal{E}$)

```

1  $success \leftarrow \text{False}$ ,  $iter \leftarrow 0$ ,  $i \leftarrow 0$ ,  $prompt\_list \leftarrow []$ ,  $\mathcal{S} \leftarrow \emptyset$ ,  $\mathcal{E} \leftarrow \emptyset$ ,  $T \leftarrow T_{\max}$ ,  $min\_makespan \leftarrow +\infty$ ;
2 while  $not\ success$  do
3    $i, MA \leftarrow \text{LLM\_leader}(\text{images}, P_A)$ ;
4    $success, baseline \leftarrow \text{BaseEvaluation}(I, MA)$ ;
5    $M \leftarrow \text{StrategyModuleExtraction}(I, MA)$ ;
6    $LLM\_probs \leftarrow \text{WeightCalculation}(LLM\_s)$ ;
7    $M^0 \leftarrow \text{LLM\_collaborators}(M, P_A, LLM\_probs, T)$ ;
8    $success, msg \leftarrow \text{ModuleEvaluation}(I, MA, M^0, baseline)$ ;
9    $LLM\_s, P_M^0 \leftarrow \text{OperatorSelection}(success, msg, P_B, M^0)$ ;
10   $\mathcal{S} \leftarrow \mathcal{S} \cup \{(M^0, P_M^0)\}$ ;
11 while  $|\mathcal{S}| < ps$  do
12    $T \leftarrow \max(0, T \cdot (1 - i/T_{\max}))$ ;
13    $LLM\_collaborators \leftarrow \text{RouletteSelection}(LLM\_s, \epsilon_{LLM})$ ;
14    $M^{i+1} \leftarrow \text{LLM\_collaborators}(M^i, P_M^i, T)$ ;
15    $success, msg, makespan \leftarrow \text{ModuleEvaluation}(I, MA, M^{i+1}, baseline)$ ;
16    $LLM\_s, P_M^{i+1} \leftarrow \text{OperatorSelection}(success, msg, M^{i+1}, P_B)$ ;
17   if  $min\_makespan > makespan$  then
18      $min\_makespan \leftarrow makespan$ ;
19    $\mathcal{S} \leftarrow \mathcal{S} \cup \{(M^{i+1}, P_M^{i+1})\}$ ;
20    $i \leftarrow i + 1$ ;
21 while  $iter < MaxIter$  do
22    $M_x^{iter}, P_M^{iter} \leftarrow \text{Sampling}(\mathcal{S}, \epsilon_M)$ ;
23    $LLM\_collaborators \leftarrow \text{RouletteSelection}(LLM\_s, \epsilon_{LLM})$ ;
24    $M^{iter+1} \leftarrow \text{LLM\_collaborators/leader}(M_x^{iter}, P_M^{iter}, T)$ ;
25    $makespan \leftarrow \text{ModuleEvaluation}(M_x^{iter+1})$ ;
26   if  $makespan < min\_makespan$  and  $M_x^{iter+1} \notin \mathcal{E}$  then
27      $min\_makespan \leftarrow makespan$ ;
28      $\mathcal{E} \leftarrow \mathcal{E} \cup \{M_x^{iter+1}\}$ ;

```

C. Description of Constraint Checking for HFJSCU

Algorithm 2: Constraint check

Input: Schedule Solution (SS), Makespan ($C_{\max}$), Instance (J)

Output: isValid, message

```

1  $\varepsilon \leftarrow \emptyset$ ;
2 if  $SS$  is empty then
3   return False, "Solution is empty";
4 if  $|SS| \neq \sum_{i=1}^N K_i$  then
5    $\varepsilon \leftarrow \varepsilon \cup \{\text{"C1"}\}$  # C1: Completeness
6 for  $i \leftarrow 1$  to  $N$  do
7   for  $j \leftarrow 2$  to  $K_i$  do
8     if  $S_{i,j}^{prep} < C_{i,j-1}$  then
9        $\varepsilon \leftarrow \varepsilon \cup \{\text{"C2"}\}$  # C2: Precedence
10 for  $k \leftarrow 1$  to  $M$  do
11    $SS_{M_k} \leftarrow \text{OperationFiltering}(M_k)$ ;
12    $SS_{M_k} \leftarrow \text{Sort}(SS_{M_k})$ ;
13   for  $p \leftarrow 2$  to  $|SS_{M_k}|$  do
14     if  $S(SS_{M_k}[p]) < C(SS_{M_k}[p-1])$  then
15        $\varepsilon \leftarrow \varepsilon \cup \{\text{"C3"}\}$  # C3: Machine Conflict
16 for  $l \leftarrow 1$  to  $C$  do
17    $Prep_{CUl} \leftarrow \text{CollaborativeUnitFiltering}(CU_l)$ ;
18    $Prep_{CUl} \leftarrow \text{Sort}(SS_{CUl})$ ;
19   for  $p \leftarrow 2$  to  $|Prep_{CUl}|$  do
20     if  $S^{prep}(Prep_{CUl}[p]) < C^{prep}(Prep_{CUl}[p-1])$  then
21        $\varepsilon \leftarrow \varepsilon \cup \{\text{"C4"}\}$  # C4: Collaborative Unit Conflict
22 for  $i \leftarrow 1$  to  $N$  do
23   for  $j \leftarrow 1$  to  $K_i$  do
24     if  $S_{i,j} < C_{i,j}^{prep}$  then
25        $\varepsilon \leftarrow \varepsilon \cup \{\text{"C5"}\}$  # C5: Internal Logic
26 if  $\max\{C_{i,j}\} > C_{\max}$  then
27    $\varepsilon \leftarrow \varepsilon \cup \{\text{"C6"}\}$  # C6: Makespan
28 if  $\varepsilon = \emptyset$  then
29   return True, "OK";
30 else
31   return False,  $\text{GenerateHint}(\varepsilon)$ ;

```

D. Pre-defined Strategy Module for Solving HFJSCU

For the job shop scheduling problem, effective basic method extensions often help in obtaining optimal solutions. The hierarchical architecture of the initialization and encoding modules guides subsequent modules to address distinct subproblems. Given the inherent complexity of the crossover and mutation module's design, a reference module is preemptively established. Therefore, the following strategy module is proposed to provide a reference for error codes and guide the optimization direction of LLM.

Encoding and Initialization: The encoding method is designed based on the sub-problems of HFJSCU. As shown in Figure S-I, it includes the operation sequence (OS), machine sequence (MS), and two cooperative unit sequences (CS). In the OS, each element represents a job, and its occurrence count in the sequence represents an operation. For example, if the fifth element is 3 and it appears for the second time, it represents $O_{3,2}$. The MS and CS are ordered according to job sequence, and their elements represent machine and collaborative unit numbers, respectively. For instance, the third element in the MS indicates that the processing machine for $O_{1,3}$ is M_2 .

The initialization method uses random initialization as an example. First, $J_i = \{n_1, n_2, \dots, n_i\}$ is traversed, and n_i is duplicated to obtain an ordered OS, with its elements randomly permuted. Next, $M_{i,j} = \{M_1, M_2, \dots, M_k\}$ is randomly selected from each element in the OS to form the MS. Finally, following the MS generation method, $C = \{1, 2, \dots, r\}$ is selected to form CS1, and $C \setminus \{t_1\}$ is selected to form CS2.

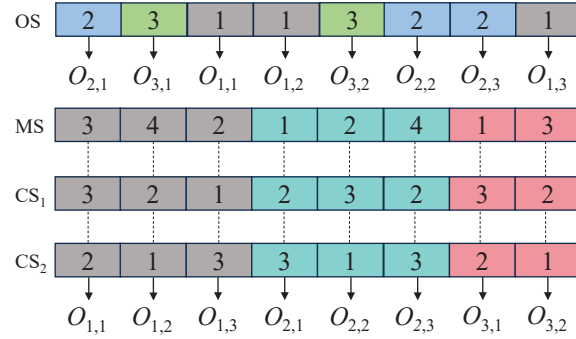


Fig. S-I. Encoding example.

Crossover: For the OS, we use Position-Based Crossover (PBX). Due to heterogeneity, Uniform Crossover (UX) is applied to the MS, and Alignment-based Crossover (ABX) is used for the CS. The specific operations are shown in Figure S-II, and the steps are as follows: 1) Generate three binary vectors, each with the same length as the OS, to select corresponding elements. 2) For PBX: When the binary vector for an operation in the first parent is 1, its position remains unchanged in the first child. The same operations from the second parent remain in their positions in the second child, while the remaining operations are sequentially filled into the remaining positions of the first child. 3) For UX: The first step is similar to PBX, where positions with a binary vector of 1 are fixed in the child. However, position dependency is not considered; the second parent fixes its remaining elements into the child's remaining positions. 3) For ABX: This can be implemented within a single solution. Collaborative units are swapped at positions where the binary vector is 1, while other positions remain unchanged.

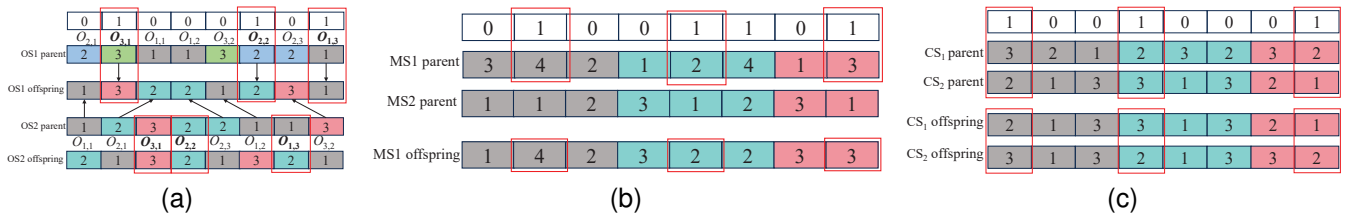


Fig. S-II. Crossover example. (a) PBX. (b) UX. (c) ABX.

Mutation: According to a probability, two different jobs in the OS undergo a swap operation. Elements in the MS are replaced with other machines that the operation can select. Elements in the CS are replaced with other collaborative units, requiring $C_{\text{new}} \in C \setminus \{C_{\text{old}}, C_{\text{excluded}}\}$.

3. Supplement to the Experimental Section

A. Hyperparameter Selection

We designed nine experiments, and the detailed data are presented in Table S-I.

TABLE S-I
COMPARISON OF OES METRICS

Experiment ID	ps	$iter$	[D,V]	$T_{\max}$	$sumOES_i$
1	40	10	[0.7, 0.3]	0.9	58.8656
2	40	15	[0.6, 0.4]	0.7	61.3286
3	40	20	[0.5, 0.5]	0.5	60.5476
4	60	10	[0.6, 0.4]	0.5	63.7042
5	60	15	[0.5, 0.5]	0.9	68.9261
6	60	20	[0.7, 0.3]	0.7	63.4934
7	80	10	[0.5, 0.5]	0.7	64.5368
8	80	15	[0.7, 0.3]	0.5	66.0380
9	80	20	[0.6, 0.4]	0.9	65.3752

B. Supplement to the Ablation Study

Comparison of data with the baseline solution obtained by I-MA, where the values in bold red font are the maximums, and the values in bold font are the minimums.

TABLE S-II
COMPARISON OF DATA WITH I-MA BASELINE SOLUTION

Instance	I-MA	MA1	MA2	LCMA1	LCMA2	LCEoMA
N10M5CU6	985	1061.5	640	685.6	645.6	636.9
N10M8CU8	897	963.1	504.9	568.8	545.4	512.7
N10M8CU8	987	1034.6	559.4	625.6	588.7	564.4
N10M10CU6	977	1021.3	546.3	617.1	560.3	561.9
N10M10CU8	1035	1069.8	565.8	633.5	576.9	560.7
N15M8CU6	1336	1452.7	742.8	828.9	755.4	739
N15M10CU8	1230	1325	655.9	753.8	652.8	633.6
N15M10CU8	1008	1003.2	547.7	611.1	564.3	549.1
N15M12CU8	996	1071	548.3	620.5	566.1	531.4
N15M12CU10	979	1049.5	556.7	617.6	544.8	548.8
N15M12CU12	1190	1280.2	628	744.2	610.7	601.3
N15M15CU6	2067	2171.3	995.6	1116	887.7	859.9
N15M15CU8	1707	1875.4	881.1	984.8	778	763.8
N15M15CU10	1759	1920.5	884.9	999.8	750.9	745.4
N20M10CU6	2149	2236.6	1013.6	1142.9	976.5	970.5
N20M15CU6	2066	2145.6	956.2	1093.5	872.8	862.6
N20M15CU10	1666	1771.2	798.4	925.6	696.4	692.6
N20M15CU15	1358	1501.5	741.5	823.6	672.6	664
N25M20CU10	1906	2019.3	912.2	1060.9	771.3	750.5
N25M20CU15	1723	1817.5	886	967.7	783.7	785.9
N30M15CU6	2983	3180.2	1346.6	1564.2	1285.7	1255.9
N30M20CU6	2999	3204.7	1413.7	1595.7	1257.3	1230.5
N30M20CU20	1767	1881.7	747.5	1046.8	736.1	732.1
N50M20CU10	3935	4052.7	1508.5	2080.8	1388.7	1370.8
N50M30CU10	3770	3889.9	1557	2073.2	1316.1	1286.1
N50M30CU15	2968	3043.2	1560.5	1659.2	1425.3	1394.7
N50M30CU20	2860	3024.5	1131.2	1630.4	943.1	951.8
N50M30CU25	2456	2661.8	1199.2	1510.4	1022.2	1015.7
N50M40CU30	2080	2221.8	841.7	1310.75	685.1	682
N50M40CU40	1618	1831.3	720	1117.8	590.5	592.6
sum	985	1061.5	640	685.6	645.6	636.9

Code Line Count Analysis: The LCEoMA framework and its variant, the LCMA2 framework, exhibit the maximum standard deviation in 7 and 5 modules, respectively. In summary, in the context of collaborative optimization between evolutionary operators and LLMs, the LCEoMA framework demonstrates the greatest diversity in 75% of the module evolutionary processes.

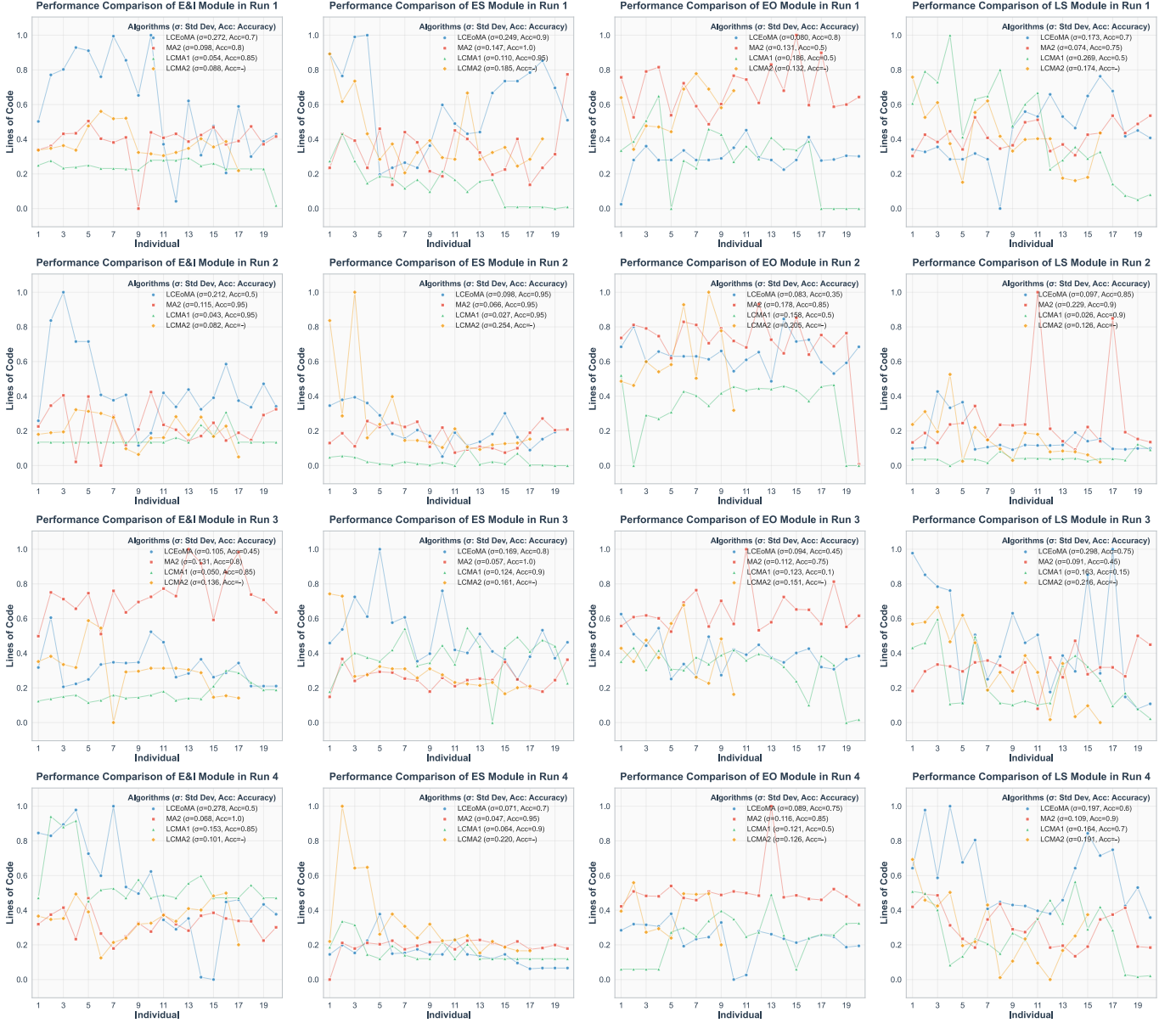


Fig. S-III. Code line count analysis.

Elo Rating System: The current experiment involves five algorithms for comparison. Consequently, for each instance, there are a total of $C(5, 2) = 10$ pairwise comparisons.

- 1) Initial Score: An initial rating score of 1000 is assigned to each algorithm.
- 2) Expected Win Rate: Based on the current Elo rating scores (R_A and R_B), the expected win rate of algorithm A, E_A , is calculated according to the following formula:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \quad (19)$$

Similarly, the expected win rate for algorithm B is $E_B = 1 - E_A$.

- 3) Result Comparison: The OES values of the two algorithms in a pair are compared. If the OES of algorithm A is greater, then $S_A = 1$ and $S_B = 0$, and vice versa. If the two are equal, then $S_A = S_B = 0.5$.

- 4) **Score Update:** The Elo rating scores of the two algorithms are updated based on the difference between the actual outcome and the expected win rate. The update formula is as follows:

$$R'_A = R_A + K \times (S_A - E_A) \quad (20)$$

where R'_A is the new score for algorithm A, and K is a constant known as the K-factor, which determines the magnitude of the score change after each comparison. A higher K-value leads to faster score changes.

CoT: Word cloud of the Chain-of-Thought component in individual module generation.

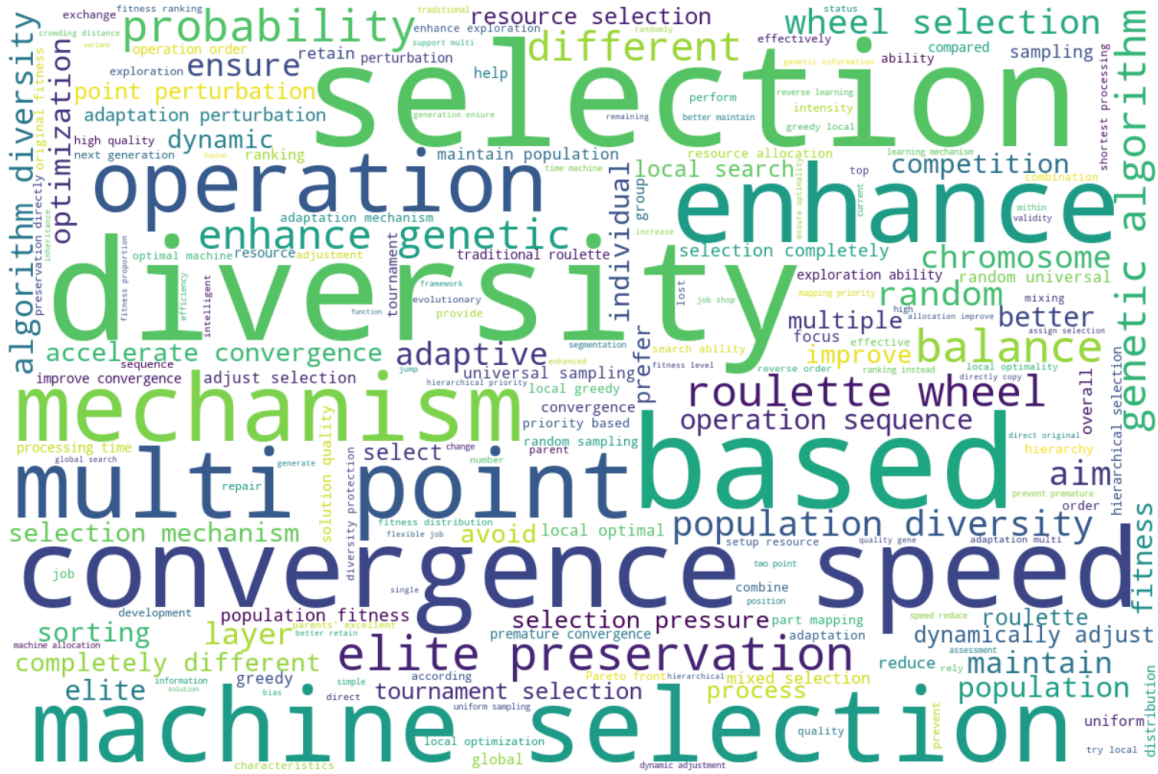


Fig. S-IV. Word cloud of the Chain-of-Thought.

C. Supplementary Comparison with Other Algorithms

Four prominent priority dispatching algorithms were selected as benchmark algorithms for a comparative analysis. These algorithms are described in detail as follows.

(1) **First-In, First-Out (FIFO)**: Prioritizes the job that has the longest waiting time at the current machine; i.e., the earliest arriving job is processed first.

(2) **Most Operations Remaining (MOPNR)**: Prioritizes the job with the highest number of remaining uncompleted operations.

(3) **Shortest Processing Time (SPT)**: Prioritizes the job that requires the shortest processing time on the current machine.

(4) **Most Work Remaining (MWKR)**: Prioritizes the job with the longest total processing time for all its subsequent remaining operations.

HFJS Dataset: The makespan results for the benchmark algorithms on the HFJS dataset are presented below.

TABLE S-III
MAKESPAN STATISTICS BASED ON THE HFJS STANDARD BENCHMARK DATASET

Instance	J-M	FIFO		MOPNR		SPT		MWKR		EoH		LCEoMA	
		best	mean	best	mean	best	mean	best	mean	best	mean	best	mean
MK01	10-6	45	47.6	42	46.8	48	54.3	44	49.3	47	50.3	41	41.5
MK02	10-6	38	41.4	38	39.7	38	46.2	38	39.5	31	39.6	27	27.6
MK03	15-8	204	216.3	204	207.7	249	278.1	204	211.8	204	228.1	204	204
MK04	15-8	70	76.6	72	76.2	71	76.4	70	72.5	78	79.1	64	65.9
MK05	15-8	185	188.3	184	188	195	213.6	186	189.1	210	214	173	173.4
MK06	10-15	96	102.2	93	98.7	74	81.5	90	100.3	75	77.2	68	69.2
MK07	20-5	186	204.5	200	210.7	192	213	199	208.8	160	186.3	144	145.6
MK08	20-10	531	536.3	523	526.4	539	624.8	524	534.2	573	586.8	523	523.5
MK09	20-10	340	357.5	323	345.8	401	438	332	352	398	404.8	325	326.6
MK10	20-15	266	282.2	256	266.6	289	342.3	259	269.2	296	321.2	227	230
09a	15-8	2180	2202	2084	2112.5	2279	2430.5	2090	2107.9	2112	2235.5	2085	2098.3
10a	15-8	2845	2890.9	2587	2709.1	2791	2891.4	2660	2715.5	2712	2796.2	2566	2603
11a	15-8	2260	2356.1	2168	2221.5	2517	2565.9	2169	2220	2319	2426.9	2134	2150.5
12a	15-8	2157	2206.5	2094	2104.9	2247	2436.5	2172	2194.9	2066	2289.4	2067	2075.4
13a	20-10	2776	2915.1	2679	2767	2864	3201.6	2765	2765	2636	2704.7	2599	2637.7
14a	20-10	2351	2399.8	2217	2276.6	2502	2625.2	2212	2239.5	2295	2450.5	2221	2231.5
15a	20-10	2312	2336.5	2190	2206.9	2583	2693.6	2175	2180.4	2449	2523.2	2186	2193.6
16a	20-10	2729	2904	2625	2739.7	2950	3194.5	2793	2831.6	2631	2692.6	2543	2560.9
17a	20-10	2386	2423.9	2230	2266.6	2396	2592.3	2211	2246	2265	2436.3	2202	2218.1
18a	20-10	2293	2346.6	2186	2205.7	2374	2590.1	2164	2178.6	2291	2470.2	2162	2168.8
lar03_1	50-60	337	349.9	335	349.6	237	243.6	331	341.1	300	324.9	229	229.1
lar03_2	50-60	577	352.7	581	352.5	409	239.1	574	343.9	231	230.7	228	234.8
lar03_3	50-60	338	356.3	333	356.1	232	235.8	334	350.3	280	295.8	231	233.6
lar03_4	50-60	592	364.5	585	353.8	415	252.6	584	352	257	265.4	243	248.6
lar03_5	50-60	349	355.9	343	350.1	230	247.9	333	353.2	320	327.3	234	235.9
lar04_1	100-60	600	599.1	589	600.6	426	415.7	599	595.6	501	544.1	417	420.3
lar04_2	100-60	353	597.5	337	603.4	244	420.1	334	601.8	485	511.2	413	418.3
lar04_3	100-60	587	614	578	615.6	415	430.7	584	615.4	477	505.2	429	429.1
lar04_4	100-60	335	596.2	328	605.3	245	427.2	341	598.9	306	527.7	424	428.2
lar04_5	100-60	403	611.2	588	604.4	421	428.4	586	604.3	427	458.4	421	426.5

HFJSCU Dataset: The makespan results for the benchmark algorithms on the HFJSCU dataset are presented below.

TABLE S-IV
MAKESPAN RESULTS ON HFJSCU DATASET

Instance	MOPNR		GA		AESSPSO		AutoV		EoH		LLMMA		LCMA	
	best	mean	best	mean	best	mean	best	mean	best	mean	best	mean	best	mean
I-1	678	714.6	976	1023.7	881	962.4	1072	1096.9	665	705.1	620	629.4	606	621.8
I-2	563	612.4	882	921	778	854.5	932	997.5	545	596.5	497	508.9	488	512.4
I-3	594	617.4	909	970.4	843	950.2	1023	1070.9	565	589.6	546	556.5	524	539.1
I-4	599	645.6	923	974.1	851	968.5	1007	1092.6	574	592	525	544.1	541	559.2
I-5	591	613	910	1010.2	908	972.8	1101	1139	582	600.3	541	563.4	521	547.3
I-6	837	888.6	1354	1432.4	1257	1321.9	1456	1528.3	720	755.8	747	764.8	718	724.6
I-7	680	733.8	1187	1260.8	1105	1220.9	1317	1381.3	670	695.2	662	677.3	597	622.1
I-8	587	667.2	950	998.9	898	957.3	1051	1095.8	575	603.9	520	535.7	501	508.8
I-9	613	655.8	980	1043.3	879	988	1027	1141.9	562	618.5	536	542.3	504	526.4
I-10	575	615.8	936	986.3	850	903.7	1023	1081.3	590	617.7	523	538.4	487	495.8
I-11	604	648.8	1082	1196.4	1000	1156.7	1277	1330.7	610	620.7	638	655.3	555	577
I-12	1108	1164.6	1979	2075.2	1877	1998.1	2261	2362.9	1076	1139.2	1051	1079.2	971	1000.5
I-13	892	948.6	1701	1830.2	1509	1666.7	1987	2054.1	850	863.3	934	961.9	803	828.8
I-14	851	950.4	1732	1842.3	1593	1785.9	1975	2038.2	635	772.1	923	955.5	747	774.4
I-15	1287	1403.6	2158	2257.2	1924	2056	2357	2417.5	1065	1080.5	1110	1147.1	938	963.8
I-16	1170	1203.8	2045	2165.9	1889	2032.4	2293	2375.5	1130	1151.2	1045	1074.6	900	964.2
I-17	775	860.8	1638	1697.6	1477	1602.4	1876	1907	739	767.5	843	879.6	662	692.7
I-18	609	662.4	1356	1434.8	1180	1342.8	1483	1597	625	644.8	723	761.5	558	570.5
I-19	1015	1067	1843	1994.8	1723	1823.3	2131	2203.4	874	890.5	993	1024.7	752	783.2
I-20	704	738.2	1626	1767.7	1513	1688.1	1866	1988	708	718.1	895	923.3	607	627
I-21	1815	1925.8	3035	3262.4	2805	2984.6	3434	3518.3	1532	1628.6	1614	1654.5	1279	1304.1
I-22	1735	1784	3126	3232.9	2905	3046.6	3376	3503.5	1646	1696.8	1595	1651.7	1299	1348.6
I-23	711	750.4	1692	1819.2	1629	1782.2	1974	2057.7	717	737.2	961	996.7	614	629.2
I-24	2179	2240.4	3830	4081	3542	3792.2	4209	4423.6	1617	1647.5	2182	2231.6	1292	1315
I-25	1853	2059.6	3709	3918.8	3308	3690.3	4114	4287.1	1658	1709.3	2060	2132	1323	1362.1
I-26	1319	1400.8	2958	3063.2	2744	2918.9	3268	3381.1	1089	1160.8	1618	1688.9	931	952.3
I-27	1102	1173.4	2958	3038.4	2734	2868.2	3276	3383.2	986	1016.2	1682	1765.7	847	886.1
I-28	918	952.4	2461	2617.3	2356	2523.6	2830	2993.5	812	849.9	1480	1540.5	755	774.8
I-29	733	827.2	2110	2186.3	2012	2162.7	2421	2508	694	744.7	1224	1287	604	627.4
I-30	633	677.6	1684	1769.4	1638	1769.9	1993	2064.8	496	609	1004	1045.4	515	526

Nemenyi test: The Nemenyi test calculates a critical difference (CD). If the difference between the average ranks of two algorithms is greater than this CD value, we consider there to be a significant difference between them.

$$CD = q_{\alpha} \sqrt{\frac{k(k+1)}{6N}} \quad (21)$$

where k represents the number of algorithms, N represents the number of instances, $\alpha = 0.05$. q_{α} is obtained from a lookup table ($q_{\alpha}(\alpha = 0.05, k = 7, df = \infty) = 2.949$).

The results for the OES_{mean} and RI_{mean} metrics of the benchmark algorithms are presented below.

TABLE S-V
RESULTS FOR THE OES_{MEAN} AND RI_{MEAN} METRICS ON THE HFJSCU DATASET

	MOPNR	GA	AESSPSO	AutoV	MA-EoH	LLMMA	LCMA	RImean (%)
N10M5CU6	0.1565	0.1092	0.1162	0.1019	0.1586	0.1777	0.1798	1.2075
N10M8CU6	0.1557	0.1036	0.1116	0.0956	0.1599	0.1874	0.1861	-0.6878
N10M8CU8	0.1622	0.1032	0.1054	0.0935	0.1699	0.18	0.1858	3.1267
N10M10CU6	0.1574	0.1043	0.1049	0.093	0.1717	0.1868	0.1818	-2.7752
N10M10CU8	0.1665	0.1011	0.1049	0.0896	0.1701	0.1812	0.1865	2.8576
N15M8CU6	0.1553	0.0964	0.1044	0.0903	0.1826	0.1805	0.1905	4.1281
N15M10CU8	0.1658	0.0965	0.0996	0.0881	0.175	0.1796	0.1955	8.15
N15M10CU8	0.1507	0.1007	0.105	0.0918	0.1665	0.1877	0.1976	5.0215
N15M12CU8	0.1567	0.0985	0.104	0.09	0.1661	0.1895	0.1952	2.932
N15M12CU10	0.1598	0.0998	0.1089	0.091	0.1593	0.1828	0.1985	7.9123
N15M12CU12	0.1739	0.0943	0.0975	0.0848	0.1818	0.1722	0.1955	7.0404
N15M15CU6	0.1694	0.0951	0.0987	0.0835	0.1732	0.1828	0.1972	7.2924
N15M15CU8	0.173	0.0897	0.0985	0.0799	0.1901	0.1707	0.1981	3.9963
N15M15CU10	0.1676	0.0865	0.0892	0.0781	0.2063	0.1667	0.2057	-0.2979
N20M10CU6	0.1457	0.0906	0.0995	0.0846	0.1893	0.1783	0.2122	10.8006
N20M15CU6	0.1648	0.0916	0.0976	0.0835	0.1723	0.1846	0.2057	10.2736
N20M15CU10	0.1713	0.0868	0.092	0.0773	0.1921	0.1676	0.2128	9.7459
N20M15CU15	0.1842	0.0851	0.0909	0.0764	0.1893	0.1603	0.2139	11.523
N25M20CU10	0.1611	0.0862	0.0943	0.078	0.1931	0.1678	0.2195	12.0494
N25M20CU15	0.1912	0.0798	0.0836	0.071	0.1965	0.1528	0.2251	12.6863
N30M15CU6	0.1514	0.0894	0.0977	0.0829	0.179	0.1762	0.2235	19.9251
N30M20CU6	0.1639	0.0904	0.096	0.0835	0.1723	0.177	0.2168	18.3508
N30M20CU20	0.1937	0.0799	0.0816	0.0707	0.1972	0.1459	0.2311	14.65
N50M20CU10	0.1489	0.0818	0.088	0.0754	0.2026	0.1495	0.2538	20.1821
N50M30CU10	0.1601	0.0841	0.0893	0.0769	0.1929	0.1546	0.242	20.3124
N50M30CU15	0.1707	0.0781	0.0819	0.0707	0.206	0.1416	0.2511	17.9618
N50M30CU20	0.1892	0.0731	0.0774	0.0656	0.2185	0.1257	0.2505	12.8026
N50M30CU25	0.1989	0.0724	0.0751	0.0633	0.2229	0.123	0.2445	8.8363
N50M40CU30	0.1937	0.0733	0.0741	0.0639	0.2152	0.1245	0.2554	15.7513
N50M50CU40	0.1944	0.0745	0.0744	0.0638	0.2163	0.126	0.2505	13.6289
mean	0.1684	0.8987	0.0947	0.0812	0.1862	0.166	0.2134	9.3128

Best Modules: In the experimental process, tens of thousands of strategy module individuals were generated. To demonstrate the advantages of the LCEoMA framework, we manually extracted the best combination of excellent modules generated during the experiment, and the synthesized algorithm is referred to as $LCMA_{best}$. Simultaneously, the best LS module was replaced with the LS in $LLMMA$, which is then referred to as $LLMMA_{best}$. The algorithms were executed independently 10 times, and the average makespan values are recorded in Table S-VI. It can be seen that, compared to Table S-IV, the makespan of both $LLMMA_{best}$ and $LCMA_{best}$ significantly decreased. However, a single-module optimization approach is less effective than an approach that optimizes all modules independently.

TABLE S-VI
COMPARISON OF ALGORITHMS AFTER BEST MODULE COMBINATION

	$LLMMA_{best}$	$LCMA_{best}$
N10M5CU6	631.6	558.3
N10M8CU6	485.5	416.4
N10M8CU8	542.5	443.7
N10M10CU6	522.6	433.7
N10M10CU8	521.7	445.9
N15M8CU6	699.8	604.1
N15M10CU8	606.2	511.2
N15M10CU8	523.2	424.6
N15M12CU8	522.6	412.5
N15M12CU10	527.7	422.5
N15M12CU12	581.9	482.2
N15M15CU6	815.8	721.7
N15M15CU8	695.2	610
N15M15CU10	721.2	597.3
N20M10CU6	931.1	766.6
N20M15CU6	834	722
N20M15CU10	670.1	540.4
N20M15CU15	621.3	485.9
N25M20CU10	726.7	586.9
N25M20CU15	635.2	506
N30M15CU6	1191.5	1007.7
N30M20CU6	1243.2	1032.5
N30M20CU20	689.1	553.6
N50M20CU10	1309.6	1055.6
N50M30CU10	1284.7	1056.4
N50M30CU15	936.6	738.6
N50M30CU20	909.3	730.1
N50M30CU25	844.6	670.6
N50M40CU30	683.2	522
N50M50CU40	597	434.7